



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 4

Team No.: 9

Project Title: Process Termination Messages

Team Members

Roll Number	Student Name	Signature
2520030386	T. Jashwanth Ram	
2520030433	J. Sai Shashank	
2520030472	A. Arun Tej	

1. Abstract

Process Termination Messages is a Linux-based Operating Systems and Systems Programming project that demonstrates how a parent process can create, execute, monitor, and identify the termination of a child process. In normal process management, users may need clear information about why and how a process ended. The proposed system uses C programming with Linux/POSIX mechanisms such as `fork()`, `waitpid()`, `exit()`, process identifiers, and termination-status macros. A parent process creates a child process; the child performs a selected task and terminates with an exit status, and the parent waits for the child and displays a meaningful termination message. The system will distinguish normal termination from termination due to other conditions and display relevant process information such as PID and exit status. The project provides a simple practical demonstration of process creation, parent-child relationships, synchronization, and process termination in Linux. The expected outcome is a small, reliable command-line program that helps users understand process termination while demonstrating core Operating Systems and POSIX system-programming concepts.

2. Problem Statement

A process may finish normally or terminate because of an error or another condition, but beginners often find it difficult to understand how the parent process detects and reports the termination of a child process. This project addresses that problem by implementing a simple Linux program in which a parent creates a child process, waits for it, checks its termination status, and displays an appropriate message. The project is directly related to Operating Systems concepts including process creation, parent-child relationships, process synchronization, exit status, and process termination.

3. Objectives

1. Demonstrate creation of a child process using `fork()`.

2. Monitor and wait for child-process completion using `waitpid()`.
3. Identify and display normal or abnormal process termination and the corresponding status.
4. Provide a simple Linux command-line demonstration of process management and termination.

4. Proposed Methodology

The project will be implemented as a command-line C program in Linux/Ubuntu. The parent process will call `fork()` to create a child process. The child will execute a simple task and terminate using a selected exit status. The parent will use `waitpid()` to wait for the child and obtain its termination status. The returned status will be examined using POSIX status macros such as `WIFEXITED()`, `WEXITSTATUS()`, and `WIFSIGNALED()`. Based on the result, the program will print a clear termination message containing relevant process information such as the child PID and termination status. Different test cases will be used to demonstrate normal and abnormal termination scenarios.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Creation – <code>fork()</code>	Creates a child process from the parent process.
<code>waitpid()</code>	Allows the parent to wait for a specific child and obtain its termination status.
<code>exit()</code>	Terminates the child process with a specified exit status.
<code>getpid()</code>	Obtains the process ID for displaying process information.
<code>WIFEXITED()</code> / <code>WEXITSTATUS()</code>	Checks normal termination and retrieves the child's exit status.
<code>WIFSIGNALED()</code>	Checks whether the child terminated because of a signal.

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520030386	T. Jashwanth Ram	Process creation and child-process implementation using <code>fork()</code> .
2520030433	J. Sai Shashank	Termination-status handling, <code>waitpid()</code> , and termination-message logic.
2520030472	A. Arun Tej	Testing, debugging, output documentation, and project report/presentation support.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system and environment for implementation and testing.
C	Programming language used to implement the process-management program.
GCC	Compiler used to build the C program.
Makefile	Build instructions for compiling the project.
Git / GitHub	Version control and project source-code management.

8. Expected Outcome

The completed project will provide a working Linux command-line program that creates a child process, waits for its completion, identifies the termination condition, and displays a clear termination message with process information and status. Test cases will demonstrate normal and abnormal termination scenarios. The project is expected to provide a simple hands-on understanding of process creation, parent-child process relationships, synchronization, exit status, and Linux/POSIX process management.

9. GitHub Repository

GitHub Repository Link: https://github.com/jashh03/OSSP_TEAM9

Repository Created:

☐ Yes ☐ No

Faculty Name and Signature

Faculty Name: Ms. Soumya Erukonda

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____